Infraestrutura de Software
Relatório: Implementação 4 (Sincronização do Metrô)

Aluno: Victor Carraly Henrique de Lima Menezes; login: vchlm
Ambiente: Linux (CachyOS), gcc 16.2.1, GNU Make 4.4.1. O runner foi executado em Linux; macOS não foi testado.
Log: 03/11/2026; sessões iniciadas às 14:54:06 e 16:53:21; validação final às 17:49:35–17:50:01 (-03).
Repositório: https://github.com/carraly/Sincronizador-Metro

1. Objetivo

O enunciado pede a sincronização de um carro e de vários passageiros (cada um uma thread) numa estação de metrô, usando somente mutex e variáveis de condição de pthreads, com um único lock por struct station, sem semáforos e sem busy-wait. Entrego sync_car.c com as quatro funções, sync_car.h com a struct, o Makefile que gera o executável sync_car, o README e este relatório. O carro só retorna quando todos os passageiros liberados já embarcaram e o carro está cheio ou a fila acabou. Também validei casos inválidos em um teste local: após a correção, os 6/6 casos retornam; não há pendência funcional conhecida, apenas ausência de teste em macOS.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Código em C para Linux, Unix ou macOS	OK	Linux: compila e roda (log). Usa apenas libc e pthreads; macOS não testado.
pthread_mutex_init/lock/unlock e pthread_cond_init/signal/wait/broadcast	OK	Inspeção de sync_car.c: as sete chamadas aparecem.
Não usar semáforos nem outras primitivas	OK	Inspeção de sync_car.c: nenhuma outra primitiva de sincronização.
No máximo um lock por struct station	OK	Único campo pthread_mutex_t: lock.
Definir struct station, station_init e as 3 funções	OK	struct em sync_car.h; funções em sync_car.c; compila sem warnings da implementação.
load_car retorna na hora sem passageiros esperando ou com 0 assentos	OK	Runner: alarmes de 1 s e 2 s no início passam; log termina em Looks good!
load_car só retorna com todos sentados e (carro cheio ou fila esgotada)	OK	Runner compara embarcados com expected em todas as linhas; sem *****.
wait_for_car bloqueia até haver carro com assento livre	OK	Predicado remaining_seats <= 0; caso "dois carros seguidos" do teste local: OK.
Embarque simultâneo: vários retornam antes de qualquer on_board	OK	broadcast em car_arrived; runner chama on_board só depois de várias threads retornarem.
Sem busy-wait	OK	Inspeção: toda espera é pthread_cond_wait dentro de while.
Makefile compila e gera o executável sync_car	OK	make limpo gera sync_car; o único warning possível vem do runner.
README com arquivos .c, como compilar/executar/testar e o SO	OK	README.md completo (commit c01e2f7).
Link do GitHub com README e relatório em PDF	OK	https://github.com/carraly/Sincronizador-Metro; README e PDF na raiz local.
Robustez a entradas inválidas (extra, não exigido)	OK	Teste local: 6/6 OK (log 17:49 e Figura 2); test_invalid.c não incluído no tar.

3. Como Reproduzir

Requisitos: gcc, make e pthreads. Manter no mesmo diretório sync_car.c, sync_car.h, sync_car-runner.c (fornecido) e o Makefile.

Compilar:

make clean

make

Executar (saída esperada: termina em Looks good!):

./sync_car

Repetir para expor corridas (cada linha deve ser Looks good!):

for i in $(seq 1 100); do ./sync_car | tail -1; done

O Makefile usa CFLAGS = -Wall -g -pthread. Com -Wextra, o único warning vem do runner (alarm_handler(int foo), parâmetro não usado), fora da implementação.

O teste local test_invalid.c não faz parte do tar: ele foi usado para validar robustez e sua evidência está no log e nas Figuras 1–2.

4. Arquitetura (resumida)

Arquivos e responsabilidades:

- **sync_car.h**: declara struct station (1 mutex, 2 variáveis de condição, 3 contadores) e os protótipos.
- **sync_car.c**: implementa station_init, station_load_car, station_wait_for_car e station_on_board.
- **sync_car-runner.c**: runner fornecido pelo professor; não faz parte da implementação.
- **Makefile**: regras de compilação, execução e limpeza.
- **README.md**: arquivos, comandos, estratégia, estrutura e ambiente.
- **evidencias.log**: sessões de terminal e testes, enviado dentro do tar.

Campos de struct station

Campo	Significado
lock	Único mutex; protege todos os campos abaixo.
car_arrived	Passageiros dormem aqui até existir carro com assento livre.
car_full_or_done	O carro dorme aqui até poder partir (alguém embarcou: reavaliar).
remaining_seats	Assentos oferecidos pelo carro ainda não ocupados. 0 significa sem carro ou carro cheio.
waiting	Passageiros que chegaram e ainda não foram liberados.
boarding	Passageiros liberados que ainda não chamaram station_on_board.

Decisões técnicas

1. Duas variáveis de condição: passageiros e carro esperam predicados diferentes, então cada broadcast/signal acorda só o grupo certo.

2. Sem flag "carro presente": remaining_seats faz esse papel e é zerado quando o carro parte; isso reduz estado e evita que o estado vaze para o próximo carro.

3. waiting e boarding separados: o passageiro atualiza ambos ao ser liberado; o carro precisa esperar boarding == 0 para permitir embarque simultâneo.

4. O carro dorme enquanto boarding != 0 || (remaining_seats != 0 && waiting > 0): é a negação de "todos sentaram e (cheio ou fila vazia)".

5. broadcast na chegada; signal em on_board quando boarding <= 0: há um único carro dormindo em car_full_or_done e o predicado continua verdadeiro sob o mesmo lock até a partida.

6. Retorno imediato quando count <= 0 ou waiting <= 0: cobre entradas inválidas sem esperar por passageiros que não existem.

7. Guarda boarding > 0 em station_on_board: uma chamada sem embarque correspondente não deixa o contador negativo nem trava o carro.

8. struct sem typedef: o runner usa struct station station; isso evita sombrear o nome station nos parâmetros.

9. Sem pthread_*_destroy: não consta no enunciado e o programa encerra logo após os testes; não há consequência prática aqui.

4.1 Implementação em detalhe

- station_init: inicializa mutex e as duas condições com _init e zera os três contadores.
- station_load_car(count): sob o lock, retorna se count <= 0 ou waiting <= 0. Senão, faz remaining_seats = count, broadcast(car_arrived) e dorme em car_full_or_done dentro de um while com o predicado acima. Ao sair, zera remaining_seats e solta o lock.
- station_wait_for_car: sob o lock, incrementa waiting e dorme em car_arrived enquanto remaining_seats <= 0. Ao sair: waiting--, remaining_seats--, boarding++, tudo sob o mesmo lock, sem brecha entre o teste e o consumo do assento.
- station_on_board: sob o lock, decrementa boarding somente se boarding > 0; se boarding <= 0, signal(car_full_or_done).

Traço de execução: 3 passageiros esperando (A, B, C), carro com 2 assentos. Estado = (waiting, remaining_seats, boarding).

Evento	Estado	Observação
Início, A, B e C dormindo	(3, 0, 0)	Todos contados em waiting.
Carro chega: remaining_seats = 2, broadcast	(3, 2, 0)	Carro dorme e solta o lock.
A passa	(2, 1, 1)	Consome assento.
B passa	(1, 0, 2)	Assentos acabaram.

Evento	Estado	Observação
C reavalia o while	(1, 0, 2)	remaining_seats <= 0: volta a dormir, ainda em waiting.
A chama on_board	(1, 0, 1)	boarding != 0: sem signal.
B chama on_board	(1, 0, 0)	boarding chegou a 0: signal.
Carro reavalia	(1, 0, 0)	0 || (0 && 1) = falso: zera remaining_seats e retorna. C fica para o próximo carro.

Teste de sanidade do predicado de dormir do carro:

(waiting, remaining, boarding)	Deve dormir?	Expressão
(5, 10, 0)	sim (carro acabou de chegar)	0 || (1 && 1) = verdadeiro
(0, 4, 3)	sim (liberados, ninguém sentou)	1 || ... = verdadeiro
(0, 4, 0)	não (fila vazia, todos sentados)	0 || (1 && 0) = falso

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Contadores + 2 condvars	Predicados derivados da especificação (partida = boarding == 0 e (cheio ou fila vazia)), um mutex.	Não houve troca: só correções de predicado, de signal, de robustez e do nome do Makefile.
S2	-	-	-
S3	-	-	-

5.2 Diário de Tentativas

Todas as linhas abaixo têm evidência no evidencias.log. O log registra a hora no início e no fim de cada sessão; para comandos intermediários, a coluna Quando indica a sessão.

#	Est.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Abrir sessão: date, whoami, pwd e script -a evidencias.log	OK	-	14:54:06	Início do log; date = 02:54:10 PM
2	S1	make clean e make com -Wall -Wextra -g -pthread	Compilou e linkou; 1 warning	unused parameter 'foo' em alarm_handler (runner, fora da implementação)	Sessão 1	Log: sync_car-runner.c:54:24: warning
3	S1	Nova sessão e repetição de make clean e make	Mesmo resultado da linha 2	-	16:53:21	Log: Script started on ... 16:53:21
4	S1	Makefile com CFLAGS = -Wall -g -pthread (sem -Wextra); recompilar	Compilou sem warnings da implementação	-	Sessão 2	Log: gcc -Wall -g -pthread -c ...
5	S1	ls para conferir os artefatos	sync_car (26k) gerado	-	Sessão 2	Log: listagem do ls
6	S1	./sync_car (execução única)	Looks good!; carros com 45, 31 e 32 assentos; o 3º levou 24	-	Sessão 2	Log: car departed ... (expected 45, 31, 24)
7	S1	for i in $(seq 1 30); do ./sync_car | tail -1; done	Todas as saídas Looks good!	-	Sessão 2	Log: sequência de Looks good!
8	S1	Mesmo laço com 100 execuções	Sem Error nem returned early no log	-	Sessão 2	Log: for i in $(seq 1 100) ...
9	S1	gcc ... -o test_invalid test_invalid.c sync_car.c (1ª tentativa)	Falhou: cc1: fatal error: test_invalid.c: No such file or directory	Arquivo ainda não estava no diretório de trabalho	Sessão 2	Log: mensagem do cc1
10	S1	Mesma compilação (2ª tentativa) e ./test_invalid	4 de 6 casos OK; 2 TRAVOU: count negativo com fila e on_board sem embarque	count < 0 passava pelo teste count == 0; remaining_seats negativo mantinha passageiros dormindo. on_board espúrio deixava boarding negativo e boarding != 0 ficava verdadeiro para sempre.	Sessão 2, antes de 17:11	Log: saída do ./test_invalid; Figura 1

#	Est.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
11	S1	Corrigir count <= 0 e guardar boarding > 0 em station_on_board; recompilar; rodar ./test_invalid e o laço de 100	6/6 casos OK; 100/100 Looks good!; commit 788e94f	-	17:49:35–17:50:01	Log final; Figura 2
12	S1	Finalizar README e revisar o relatório	README completo; commit c01e2f7	-	17:58:48	git log; README.md

A linha 10 é o bug que mais pesou na semana: o log mostra o travamento e o resumo da hipótese fica nesta linha. A linha 11 é a correção que foi validada no log final; a Figura 1 mostra a saída antiga e a Figura 2 mostra os 6 casos OK.

5.3 Dificuldades e o que não foi implementado

Corrigidas antes da primeira compilação registrada (sem trecho no log, portanto listadas apenas como contexto, não como tentativas comprovadas):

- Primeira versão de station_load_car com laço vazio (busy-wait) e sem mutex.
- Predicado do carro com conectivos trocados (&& e || invertidos): o carro retornava cedo ou nunca dormia. Resolvido com a negação da condição de partida e a tabela de sanidade da seção 4.1.
- Condição do signal em station_on_board copiada do while do carro (invertida): o carro nunca acordava. Resolvido derivando o signal de boarding <= 0.

Corrigidas com evidência no log: count negativo em station_load_car e station_on_board sem passageiro liberado. A versão final retorna imediatamente para count <= 0 e só decrementa boarding se ele for positivo; os 6 casos do teste local passaram.

Não ficou pendência funcional. O único limite é não ter testado em macOS; o runner e o teste local foram executados apenas em Linux/CachyOS.

6. Evidências

6.1 Log automático

O arquivo evidencias.log foi gravado com script -a evidencias.log e está incluído no tar. Ele tem 59.392 bytes e 939 linhas; as sessões principais começam em 14:54:06 e 16:53:21, e a validação final vai de 17:49:35 a 17:50:01. O prompt colorido do terminal deixa sequências de escape ANSI, que o tornam ruidoso para leitura direta; os trechos abaixo foram transcritos sem essas sequências.

Trecho A: compilação (sessão 2):

```
gcc -Wall -g -pthread -c sync_car-runner.c
gcc -Wall -g -pthread -c sync_car.c
gcc -Wall -g -pthread -o sync_car sync_car-runner.o sync_car.o
```

Trecho B: execução única e laço de 100:

```
car entering station with 45 free seats
car departed station with 45 new passenger(s) (expected 45)
car entering station with 31 free seats
car departed station with 31 new passenger(s) (expected 31)
car entering station with 32 free seats
car departed station with 24 new passenger(s) (expected 24)
Looks good!
for i in $(seq 1 100); do ./sync_car | tail -1; done
100 linhas Looks good!, sem Error nem returned early
```

Trecho C: ./test_invalid final (17:49):

```
count negativo com fila                          OK (retornou)
count 0 com fila                                 OK (retornou)
count negativo sem fila                          OK (retornou)
count maior que a fila (limite)                  OK (retornou)
station_on_board sem passageiro liberado         OK (retornou)
dois carros seguidos (3 esperando, 2+2 assentos) OK (retornou)
```

6.2 Prints

Erro principal:

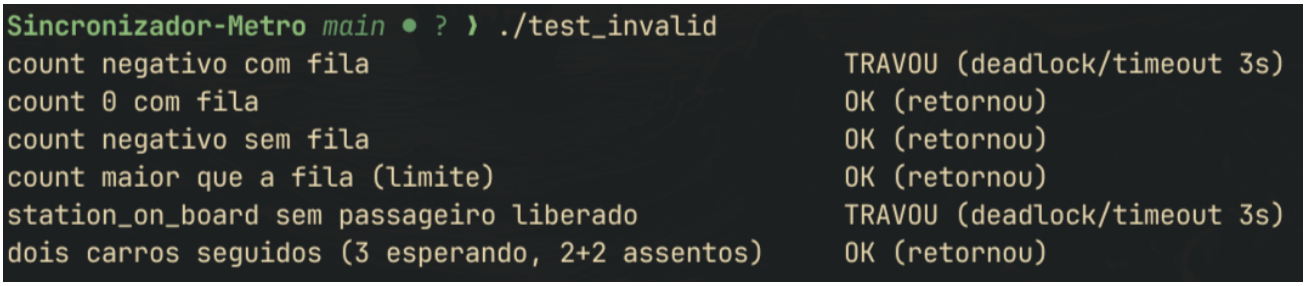


Figura 1: Saída do ./test_invalid antes da correção, com 2 de 6 casos TRAVOU (log, 17:11; captura às 17:25).

Validação final:

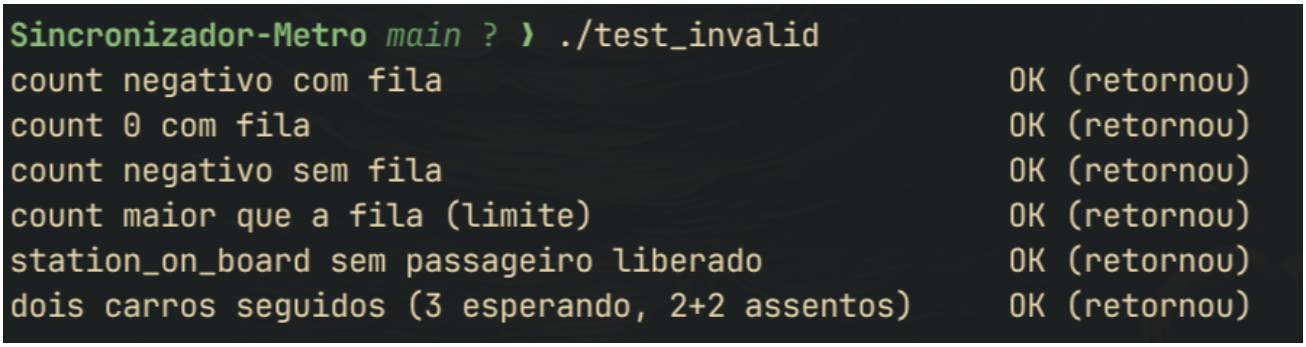


Figura 2: Validação final: os 6 casos do ./test_invalid retornam OK após a correção (captura às 18:12; execução equivalente registrada no log às 17:49).

7. Uso de IA

Onde usei IA: utilizei Claude e opencode como revisores. A IA auxiliou na localização dos casos inválidos e rodou testes para verificar o funcionamento. As decisões finais, a correção e a validação manual foram minhas.

Prompts principais:

- "Sem fazer nada por mim, me ajude a entender essa atividade e as funções necessárias."
- Envio de sync_car.h, sync_car.c e Makefile para revisão, pedindo que procurasse possíveis erros sem entregar o código pronto.
- "Faça os testes para casos inválidos."

O que validei manualmente e como:

Compilei do zero com make; executei o runner uma vez e em laços de 30 e 100; compilei e executei o test_invalid; li o log e comparei as saídas; conferi o predicado do carro com a tabela de três estados e com o traço da seção 4.1; após a correção, os 6 casos retornam OK.

8. Reflexão Final

A parte que mais me custou foi escrever o predicado do carro: duas versões erraram os conectivos e uma terceira copiou para o signal a condição do while, o que travava o programa. O que funcionou foi escrever primeiro a condição de partida em português (todos sentaram e (cheio ou ninguém esperando)), negar com De Morgan e validar com uma tabela de três estados. As decisões que mais pesaram foram separar waiting de boarding (para permitir embarque simultâneo), zerar remaining_seats ao partir e usar duas variáveis de condição. O teste de casos inválidos mostrou que o runner sozinho não cobre o contrato: count negativo e on_board sem embarque travavam o programa, e eu só descobri isso depois de o runner já passar. Na próxima vez, escreveria os casos limite e inválidos junto com as funções, e não ao final, e conferiria o nome do Makefile contra os arquivos reais antes de compilar.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim (log: make clean e make)
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim (Figuras 1 e 2)
Testei pelo menos um caso limite e um caso inválido	Sim (runner; test local 6/6; log)

Item	Confirmado?
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim
Rodei novamente com a versão final de sync_car.c e acrescentei ao log	Sim (100/100 Looks good!; 6/6 OK)
Link do GitHub com README e relatório em PDF	OK: README e PDF na raiz local

10. Se eu tivesse mais 2 horas

- Testes: rodar o runner com -fsanitize=thread e com 1.000 execuções para procurar data races e aumentaria a cobertura de chegadas de passageiros enquanto o carro ainda está na estação.
- Automação: criar um script que compare a saída do runner byte a byte com um gabarito e rode o test_invalid em CI, em vez de conferir manualmente.
- Robustez: cobrir mais casos de count negativo, count maior que a fila, dois carros seguidos e on_board sem embarque; manter o test_invalid.c versionado como teste auxiliar.
- Organização: adicionar alvo all ao Makefile, limpar as sequências ANSI do log e documentar no README os comandos de teste e os tempos esperados.

Regras de Integridade

Todos os resultados citados vêm do evidencias.log anexado, do código entregue ou das capturas das Figuras 1 e 2. Itens de contexto sem trecho no log estão marcados como tal; não há evidência fabricada.

Apêndice A - Código final

sync_car.h

```
#ifndef SYNC_CAR_H
#define SYNC_CAR_H

#include <pthread.h>

struct station{
    pthread_mutex_t lock;
    pthread_cond_t car_arrived;
    pthread_cond_t car_full_or_done;
    int remaining_seats;
    int waiting;
    int boarding;
};

void station_init(struct station *station);
void station_load_car(struct station *station, int count);
void station_wait_for_car(struct station *station);
void station_on_board(struct station *station);

#endif
```

sync_car.c

```
#include "sync_car.h"
#include <pthread.h>

void station_init(struct station *station){
    pthread_mutex_init(&station->lock, NULL);
    pthread_cond_init(&station->car_arrived, NULL);
    pthread_cond_init(&station->car_full_or_done, NULL);
    station->remaining_seats = 0;
    station->waiting = 0;
    station->boarding = 0;
}

void station_load_car(struct station *station, int count){
    pthread_mutex_lock(&station->lock);
```

```
    if (count <= 0 || station->waiting <= 0) {
        pthread_mutex_unlock(&station->lock);
        return;
    }

    station->remaining_seats = count;
    pthread_cond_broadcast(&station->car_arrived);
    while (station->boarding != 0 || (station->remaining_seats != 0 && station->waiting > 0)) {
        pthread_cond_wait(&station->car_full_or_done, &station->lock);
    }

    station->remaining_seats = 0;
    pthread_mutex_unlock(&station->lock);
}

void station_wait_for_car(struct station *station){
    pthread_mutex_lock(&station->lock);
    station->waiting += 1;

    while (station->remaining_seats <= 0) {
        pthread_cond_wait(&station->car_arrived, &station->lock);
    }

    station->waiting -= 1;
    station->remaining_seats -= 1;
    station->boarding += 1;

    pthread_mutex_unlock(&station->lock);
}

void station_on_board(struct station *station){
    pthread_mutex_lock(&station->lock);
    if (station->boarding > 0) {
        station->boarding -= 1;
    }

    if (station->boarding <= 0) {
        pthread_cond_signal(&station->car_full_or_done);
    }

    pthread_mutex_unlock(&station->lock);
}
```

Makefile

```
CFLAGS = -Wall -g -pthread

sync_car: sync_car-runner.o sync_car.o
	gcc $(CFLAGS) -o sync_car $^

%.o: %.c sync_car.h
	gcc $(CFLAGS) -c $<

run: sync_car
	./sync_car

clean:
	rm -f *.o sync_car

.PHONY: run clean
```